

Automated Produce Scanner with Real-Time Pricing and Packaging

Real-Time Pricing and Packaging System
Portfolio / Technical Documentation Edition

Architecture of the Device

Central Processing Unit	Raspberry Pi 4 Model B (4 GB RAM)
Computer Vision / ML	Raspberry Pi Camera Module V2 + Edge Impulse
Primary Software	Python + OpenCV + Edge Impulse inference
Primary Functions	Produce detection, weighing, pricing, barcode generation, and automated packaging

Acknowledgement

The successful completion of this project would not have been possible without the collective effort, knowledge, cooperation, and support of the individuals who contributed throughout its development.

First and foremost, I would like to express my sincere gratitude to my colleagues and fellow engineers, **Maria Jeni Louise Brillas, Jean Rose Jamellano, Jeston Roem Lacson, and Kane Maxine Aesha M. Magan**, for their cooperation, dedication, ideas, and valuable contributions to the fulfillment of this project. The knowledge and skills that we shared throughout the development process played an important role in overcoming challenges and bringing the project to completion.

I would also like to extend my heartfelt appreciation to my **family, mentors, friends, and other individuals** who provided encouragement, guidance, assistance, and support throughout our academic journey and the development of this project. Their continued support served as a source of motivation, especially during challenging stages of the project.

My sincere gratitude is also extended to the **faculty and staff of Carlos Hilado Memorial State University**, particularly those who provided us with the knowledge, guidance, resources, and opportunities necessary to develop and demonstrate the skills we acquired throughout our years of study. Their instruction and mentorship contributed greatly to our growth as aspiring engineers.

Above all, I give my deepest gratitude to **Almighty God** for His guidance, strength, wisdom, and protection throughout the entire process. His blessings gave us the perseverance and determination to overcome the challenges we encountered and successfully complete this project.

To everyone who, in one way or another, contributed to the completion of this work, **thank you sincerely**. This achievement would not have been possible without your support and contributions.

May God bless us all.

1. Architecture Overview

The Automated Produce Scanner with Real-Time Pricing and Packaging is an embedded retail automation prototype centered on a Raspberry Pi 4 Model B. The architecture integrates computer vision, machine-learning inference, weight sensing, local database pricing, user-interface display, barcode generation, and automated packaging into one coordinated system.

The Raspberry Pi acts as the central processing unit. It receives RGB image data from the Raspberry Pi Camera Module V2 and weight data from the load cell through the HX711 amplifier. The image is preprocessed and analyzed by an Edge Impulse inference model deployed locally on the Raspberry Pi. The recognized produce class is associated with a unit price stored in the local database. The measured weight and price are then combined to calculate the transaction amount in real time.

The resulting information is presented through the LCD display. Transaction data is also sent to the thermal printer for barcode generation, while the packaging subsystem performs automated heat sealing after transaction confirmation.

2. System Architecture

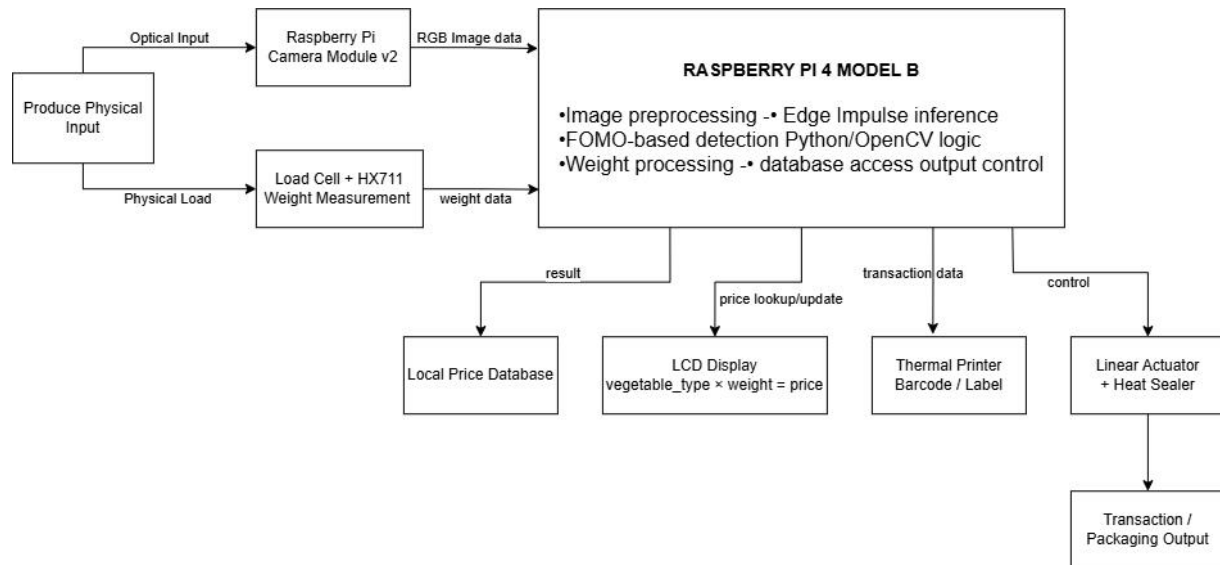


Figure 1. Original portfolio representation of the device's functional architecture.

The architecture is organized around the Raspberry Pi as the central processing and control node. The camera and weighing system provide the primary sensing inputs, while the local price database supplies the pricing information required for real-time transaction computation. The processing node coordinates the user display, barcode printing, and automated sealing outputs.

3. Hardware Architecture

Component	Architectural Role
Raspberry Pi 4 Model B	Central processing unit; coordinates inference, sensor processing, database interaction, calculations, and peripheral control.
Raspberry Pi Camera Module V2	Captures produce images and supplies RGB image data for detection.
Load Cell	Measures the physical weight of the produce placed on the weighing platform.
HX711	Amplifies and converts the load-cell signal into digital weight data for processing.
LCD Module	Displays produce type, detection information, weight, and price to the user.
Thermal Printer	Generates and prints transaction/barcode labels.
Linear Actuator	Provides mechanical movement for the automated sealing process.
Heat Sealer	Seals the produce packaging after transaction confirmation.
Buck Converter	Steps down a higher DC voltage to a lower regulated level for supported components.
Terminal Block	Organizes and secures wiring connections.
Raspberry Pi Flex Cable	Connects the Camera Module V2 to the Raspberry Pi CSI interface.

The thesis also lists a stepper motor drive as a possible component for future mechanical control; it is explicitly noted as not being part of the main implemented setup. It should therefore be described as a future/optional component rather than a core implemented subsystem.

4. Vision and Machine-Learning Architecture

The vision subsystem begins with image acquisition from the Raspberry Pi Camera Module V2. The captured image is passed to the Raspberry Pi, where image preprocessing is performed before local Edge Impulse inference. The thesis describes the trained model as a FOMO-based object detection model and states that the deployment package was exported for Linux AArch64 offline inference with INT8 quantization.

- Image acquisition through Raspberry Pi Camera Module V2.
- Image preprocessing, including resizing and normalization.
- Local Edge Impulse inference on the Raspberry Pi.

- FOMO-based object detection.
- Output of predicted class label and confidence score.
- Detection of red onion, garlic, marble potato, and the empty/no-produce condition.
- Detection logic for multiple/assorted produce to prevent an invalid transaction.

Note: The thesis contains two different input-size references: the original block diagram shows 96×96 preprocessing, while the explanatory text states 160×160 . This document intentionally does not select one as the definitive deployed value. The actual Edge Impulse project/model configuration should be checked before publishing a final technical specification.

5. Weight Measurement Architecture

The weighing subsystem provides the quantitative input required for price computation. The load cell senses the produce weight and the HX711 provides the signal-conditioning and digital conversion interface used by the Raspberry Pi. A tare function is included to zero the weighing system before measurement.

Processing relationship:

Produce $\rightarrow$ Load Cell $\rightarrow$ HX711 $\rightarrow$ Raspberry Pi $\rightarrow$ Weight Data

6. Pricing and Database Architecture

After the produce is classified, the system retrieves the corresponding unit price from the local price database. The measured weight and unit price are then combined to calculate the total transaction cost in real time. An administrator interface allows authorized personnel to modify vegetable prices stored in the database without retraining the image-classification model.

Classification + Weight + Database Price $\rightarrow$ Real-Time Total Price

7. User Interface and Transaction Outputs

7.1 LCD Display

The LCD provides the user-facing display for the recognized produce, detection information, measured weight, and calculated price.

7.2 Thermal Barcode Printer

The transaction information is forwarded to a thermal printer to generate a scannable barcode/label. The thesis describes barcode records as being matched with corresponding transaction information stored in the database.

7.3 Automated Packaging

After transaction confirmation, the Raspberry Pi controls the packaging mechanism. A linear actuator provides the mechanical movement needed to operate the sealing mechanism, while the heat sealer seals the package.

8. Software Architecture

Software Layer	Function
Camera / Image Handling	Python and OpenCV/system-level camera control.
Image Preprocessing	Preparation of captured image data for inference.
ML Inference	Edge Impulse model running locally on Raspberry Pi.
Classification Logic	Interprets class label and confidence; checks for valid single-produce input.
Weight Processing	Reads and processes load-cell/HX711 measurements.
Database Logic	Retrieves and updates unit prices in the local database.
Transaction Logic	Combines classification, weight, and price to calculate the transaction value.
Output Control	Updates LCD, triggers barcode printing, and controls packaging hardware.

9. End-to-End Data Flow

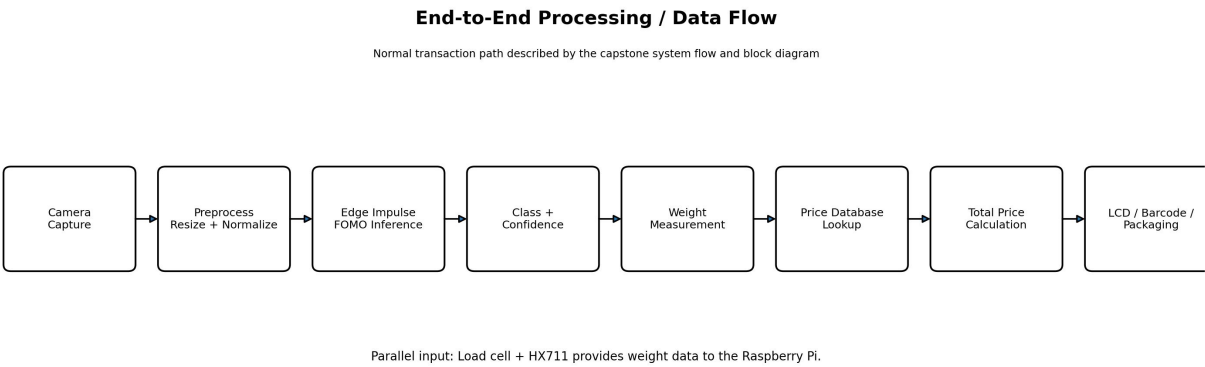


Figure 2. Simplified end-to-end data flow.

The normal transaction begins when a user places a produce item on the weighing platform. The camera captures the item, the system performs image detection, and the load cell provides the weight. If a valid single-produce

classification is obtained, the system retrieves the current unit price, calculates the total, displays the result, prints the barcode, and performs the packaging operation after confirmation.

10. Error Handling and Validation

10.1 No Produce Detected

When the platform is empty or no valid produce is detected, the system waits for a valid input instead of continuing to weighing and pricing.

10.2 Multiple / Assorted Produce Detected

When multiple vegetables are detected simultaneously, the system enters an error state and displays an 'Error: Assorted Detected' message. Processing remains in the error loop until a valid single-vegetable input is detected. This prevents mixed-produce transactions.

10.3 Tare / Reset

The tare control allows the weighing system to be reset to zero before a measurement.

11. Administrator Architecture

Administrator → Price Management Interface → Local Database → Updated Unit Price → Pricing Logic

The administrator path is separate from the customer transaction path. Authorized personnel can update vegetable prices stored in the local database. According to the thesis, price changes are reflected in real-time pricing calculations without requiring retraining or recalibration of the image-classification model.

12. Physical Device Architecture

The physical architecture should be documented separately from the functional block diagram. The camera is mounted above the produce platform, while the weighing platform, display, printing mechanism, and sealing mechanism are arranged to support the intended transaction sequence. The thesis describes the system layout as including the load cell, camera, sealing mechanism, and display units, with the modules assembled and integrated according to the design layout.

For a public GitHub version, use photographs of the actual prototype and create an original annotated physical-layout diagram. Do not copy the university's thesis figure directly.

13. Architecture Limitations

- The implemented model is limited to the trained produce categories: red onion, garlic, and marble potato.
- Image-detection performance is sensitive to lighting and image-acquisition conditions.
- The image model identifies produce type but does not determine whether a vegetable is overripe or damaged.
- The weighing system can exhibit minor fluctuations, although the reported results remained within an acceptable tolerance.
- The packaging mechanism required calibration and hardware adjustment to improve sealing consistency.
- The system is primarily a standalone/local prototype rather than a cloud-connected retail infrastructure.

14. Portfolio Notes and Technical Accuracy

This document is a portfolio-oriented technical adaptation of the capstone architecture. It is not a reproduction of the full thesis. The diagrams are newly drawn for documentation purposes.

Before publishing the architecture publicly, verify the exact final hardware configuration, GPIO/pin assignments, model input dimensions, database technology, and the portions of source code that you are personally authorized to publish. The thesis supports the functional relationships documented here, but it does not by itself establish ownership or publication permission for every underlying project artifact.

15. Source Basis

Primary source: Submitted capstone thesis, Automated Produce Scanner with Real-Time Pricing and Packaging. This architecture document was prepared from the thesis sections covering system design, methodology, block diagram, flowchart, hardware/software components, implementation, findings, conclusions, and limitations.